PYTHON OS MODULE

What is the os Module?

The `os` module in Python provides a way to interact with the **operating system**.
It allows you to perform tasks like creating files/folders, navigating directories, and managing environment variables.

1. os.getcwd() => Get Current Working Directory

```
import os
print(os.getcwd())
```

Explanation:
This function shows where your Python script is running from.

2. os.mkdir() => Create a New Directory

```
os.mkdir("new_folder")
```

Explanation:
Creates a folder named new_folder in the current directory.
If it already exists, it will give an error.

3. os.makedirs() => Create Directory with Subfolders

```
os.makedirs("parent/child/grandchild")
```

Explanation:
Creates all folders in the path if they don't exist.
This will create parent, then child inside it, and grandchild inside that.

---

4. os.chdir() => Change Directory

```
os.chdir("C:/Users/Username/Documents")
print(os.getcwd())
```

Explanation:
This changes the current working directory to the one you provide.
After that, os.getcwd() will show this new location.

---

5. os.listdir() => List Files and Folders in Directory

```
files = os.listdir()
print(files)
```

Explanation:
Lists everything (files and folders) in the current directory.
You can also pass a path like os.listdir("C:/Users").

---

6. os.rename() => Rename File or Folder

```
os.rename("old_name.txt", "new_name.txt")
```

Explanation:
Changes the name of a file or folder.

---

7. os.remove() => Delete a File

```
os.remove("unwanted.txt")
```
Explanation:
Deletes the file named unwanted.txt.
Make sure it's a file, not a folder.

---

8. os.rmdir() => Remove an Empty Folder

```
os.rmdir("empty_folder")
```
Explanation:
Removes a folder, but it must be **empty**.
Use os.removedirs() for recursive removal.

---

9. os.path.exists() => Check if Path Exists

```
if os.path.exists("data.txt"):
    print("File exists")
else:
    print("Not found")
```
Explanation:
Checks whether the file or folder is present or not.

---

10. os.path.join() => Join Two Paths

```
folder = "C:/Users"
file = "notes.txt"
full_path = os.path.join(folder, file)
print(full_path)
```
Explanation:
It joins folder and file path safely.
Works on both Windows (\) and Linux/Mac (/).

---

11. os.path.isfile() & os.path.isdir()

```
print(os.path.isfile("file.txt"))   # True if it is a file
print(os.path.isdir("foldername"))  # True if it is a folder
```

---

12. os.environ and os.getenv()

```
print(os.environ)  # Shows all environment variables

# Get specific one
print(os.getenv('PATH'))
```
Explanation:
Used to read system environment variables like PATH, HOME, etc.

LOCAL SCOPE VS GLOBAL SCOPE IN PYTHON

1. GLOBAL SCOPE

- **Definition**: A variable with global scope is defined outside all functions and is accessible throughout the program.

- **Lifespan**: It exists as long as the program runs.
- **Access**: Can be accessed from inside and outside functions.

Example:

```
x = 10  # Global variable

def show():
    print(x)  # Accessing global variable

show()
print(x)  # Also accessible here
```

2. LOCAL SCOPE

- **Definition**: A variable with local scope is defined inside a function and can only be used inside that function.
- **Lifespan**: It exists only while the function is executing.
- **Access**: Not accessible outside the function.

Example:

```
def show():
    y = 5  # Local variable
    print(y)

show()
# print(y)  #  This will cause an error (y is not defined here)
```

Using Global Keyword

If you want to **modify** a global variable inside a function, you must use the `global` keyword.

Example:

```
x = 10

def update():
    global x
    x = 20  # Modifies the global variable
    print("Inside:", x)

update()
print("Outside:", x)
```

PYTHON FILE HANDLING MODES

Basic Syntax to Open a File

```
file = open("filename.txt", "mode")
```

File Modes in Python

Mode	Name	Description
`'r'`	**Read**	Opens the file for reading. File **must exist**.

Mode	Name	Description
`'w'`	**Write**	Opens the file for writing. **Overwrites** the file if it exists, or creates a new file.
`'a'`	**Append**	Opens the file for appending. Adds content to the end without deleting existing data.
`'x'`	**Exclusive Creation**	Creates a new file. Fails if the file already exists.
`'t'`	**Text Mode**	Default mode. File is handled as text.
`'b'`	**Binary Mode**	File is handled as binary (images, videos, etc.).

Combined Modes

Mode	Description
`'rt'`	Read text (default)
`'rb'`	Read binary
`'wt'`	Write text (overwrites)
`'wb'`	Write binary (overwrites)
`'at'`	Append text
`'ab'`	Append binary
`'r+'`	Read and write (file must exist)
`'w+'`	Write and read (file is overwritten)
`'a+'`	Append and read

Examples

1. Reading a File

```
f = open("data.txt", "r")
content= f.read())
print(content)
f.close()
```

2. Writing to a File

```
f = open("data.txt", "w")
f.write("Hello, world!")
f.close()
```

3. Appending to a File

```
f = open("data.txt", "a")
f.write("\nNew line added.")
f.close()
```

4. Read + Write

```
f = open("data.txt", "r+")
data = f.read()
print(data)
f.write("\nExtra line.")
f.close()
```

Tips

Always use `close()` to close files or use:

```
with open("file.txt", "r") as f:
    data = f.read()
```

with automatically closes the file.

SOME MORE METHODS IN FILE HANDLING

readlines() method:

The readline() method reads a single line from the file. If we want to read multiple lines, we can use a loop.

```
f = open('myfile.txt', 'r')
while True:
    line = f.readline()
    if not line:
        break
    print(line)
```

The readlines() method reads all the lines of the file and returns them as a list of strings.

writelines() method:

The writelines() method in Python writes a sequence of strings to a file. The sequence can be any iterable object, such as a list or a tuple.
Here's an example of how to use the writelines() method:

```
f = open('myfile.txt', 'w')
lines = ['line 1\n', 'line 2\n', 'line 3\n']
f.writelines(lines)
f.close()
```

This will write the strings in the lines list to the file myfile.txt. The \n characters are used to add newline characters to the end of each string.
Keep in mind that the writelines() method does not add newline characters between the strings in the sequence. If you want to add newlines between the strings, you can use a loop to write each string separately:

```
f = open('myfile.txt', 'w')
lines = ['line 1', 'line 2', 'line 3']
for line in lines:
    f.write(line + '\n')
f.close()
```

seek() function:

The seek() function allows you to move the current position within a file to a specific point. The position is specified in bytes, and you can move either forward or backward from the current position. For example:

```
with open('file.txt', 'r') as f:
  # Move to the 10th byte in the file
  f.seek(10)

  # Read the next 5 bytes
  data = f.read(5)
```

tell() function:

The tell() function returns the current position within the file, in bytes. This can be useful for keeping track of your location within the file or for seeking to a specific position relative to the current position. For example:

```
with open('file.txt', 'r') as f:
  # Read the first 10 bytes
  data = f.read(10)

  # Save the current position
  current_position = f.tell()

  # Seek to the saved position
  f.seek(current_position)
```

truncate() function:

When you open a file in Python using the open function, you can specify the mode in which you want to open the file. If you specify the mode as 'w' or 'a', the file is opened in write mode and you can write to the file. However, if you want to truncate the file to a specific size, you can use the truncate function.
Here is an example of how to use the truncate function:

```
with open('sample.txt', 'w') as f:
  f.write('Hello World!')
  f.truncate(5)

with open('sample.txt', 'r') as f:
  print(f.read())
```

LAMBDA FUNCTION IN PYTHON

A **lambda function** is an **anonymous (nameless) function** defined using the `lambda` keyword.
It is used for creating **small, one-line functions** without a name.
Typically used when a short function is needed for a short period of time.

Example 1: Add two numbers

```
add = lambda a, b: a + b
print(add(5, 3))  # Output: 8
```

Example 2: Square of a number

```
square = lambda x: x * x
print(square(4))  # Output: 16
```

MAP FILTER REDUCE IN PYTHON

1. **`MAP()`** – APPLY A FUNCTION TO EACH ITEM

```
def square(x):
    return x * x

numbers = [1, 2, 3, 4]
squared = list(map(square, numbers))
print(squared)  # Output: [1, 4, 9, 16]
```

2. **`FILTER()`** – KEEP ITEMS THAT MATCH A CONDITION

```
def is_even(x):
    return x % 2 == 0

numbers = [1, 2, 3, 4, 5]
evens = list(filter(is_even, numbers))
print(evens)  # Output: [2, 4]
```

3. **`REDUCE()`** – REDUCE TO A SINGLE VALUE

```
from functools import reduce

def multiply(x, y):
    return x * y

numbers = [1, 2, 3, 4]
product = reduce(multiply, numbers)
print(product)  # Output: 24
```

Summary Table (Using Named Functions):

Function	Custom Function Example	What It Does	Output Example
`map()`	`square(x)`	Transforms each item	`[1, 4, 9, 16]`
`filter()`	`is_even(x)`	Filters based on condition	`[2, 4]`
`reduce()`	`multiply(x, y)`	Combines all items into one value	`24`

"IS" VS "==" IN PYTHON

1. **`==` (Equality Operator)**

**Checks if the values of two variables are equal.
It compares data/content stored inside the objects.

Example:

```
a = [1, 2, 3]
b = [1, 2, 3]

print(a == b)  # Output: True
```

Even though `a` and `b` are two different lists (stored in different memory locations), their values are the same, so `==` returns True.

2. is (Identity Operator)

**Checks if two variables refer to the same object in memory.
It compares the memory address (object identity).

```
a = [1, 2, 3]
b = [1, 2, 3]

print(a is b)  # Output: False
```

`a` and `b` have the same values, but they are different objects in memory. So `is` returns False.

Special Case – Small Integers & Strings

Python caches small integers and strings, so sometimes `is` and `==` both return `True`:

```
x = 5
y = 5
print(x is y)  # Output: True (because of caching)

s1 = "hello"
s2 = "hello"
print(s1 is s2)  # Output: True
```

Summary Table:

Operator	Checks for	Compares	Example Output	Use When
`==`	Equality	Values	`True`	You care about content
`is`	Identity (same object)	Memory addresses	`False`	You care about same object

Final Tip:

Use `==` when comparing values (numbers, lists, strings).
Use `is` when checking if two variables point to the same object (like `None`, singletons).